Basics of Digital Image Processing

Course Instructor: **Prof. Soumen Bag**

Department of Computer Science and Engineering
IIT (ISM) Dhanbad

Class Information

Course: Digital Image Processing | NCSH302 | 0-0-3

Class Timing: Tuesday: 9:00 AM - 11:50 AM

Class Venue: NLHC Lab-1

Google Class Invitation link:

https://classroom.google.com/c/ODU1NjQ1MTk3MDc2?cjc=zs3txgkw

Laboratory Plan

Laboratory Plan

Sl. No.	Laboratory	No. of Hours
1.	**Introduction to MATLAB and Python:** Read Image, Display Image, Write Image, Image Color Conversion, Image Cropping, Image Scaling, Image Splitting and Merging.	3
2.	**Basics of Image Processing:** Image Sampling and Quantization, Image Representation, Pixel Geometry.	6
3.	**Image Enhancement:** Contrast Enhancement, Histogram Processing, Spatial Domain Filtering, Edge Sharpening, Frequency Domain lowpass and highpass Filtering.	6

Laboratory Plan

Sl. No.	Laboratory	No. of Hours
4.	**Image Restoration:** Image Restoration Filtering, Notch Filter, Image Estimation, Geometric Transformation.	3
5.	**Image Morphology:** Fundamental Operations; Morphological Algorithms.	3
6.	**Image Segmentation:** Pixel-based Segmentation, Adaptive Thresholding, Region-based Segmentation, Point, Line, and Edge detection, Canny Edge Detector, Hough Transform.	6

Laboratory Plan

Sl. No.	Laboratory	No. of Hours
7.	**Image Compression:** Lossless Compression (Run-length Coding, Shannon-Fano Coding, Huffman Coding, Arithmetic Coding), Lossy Compression (BTC, VQ).	6
8.	**Image Representation and Description:** Binary Tree and Quad Tree Coding, Freeman Chain Coding, Thinning, Boundary Descriptors, Regional Descriptors, Topological Descriptors, Relational Descriptors.	3
9.	**Multiresolution Analysis and Wavelet:** Pyramidal Coding; Subband Coding; Application of Wavelets.	3
10.	**End Semester Examination** Syllabus: Questions will be given from the topics covered throughout labs.	3

Evaluation Pattern

Continuous Evaluation	Viva-voce	End-Semester Examination
Marks: 30	**Marks: 20**	**Marks: 50**
Date: Throughout the entire lab sessions	**Date:** 10 November, 2026	**Date:** 17 November, 2026
Syllabus: As per the laboratory plan	**Syllabus:** This will be informed by the respective Course Instructor.	**Syllabus:** This will be informed by the respective Course Instructor.

Text and Reference Books

Text and Reference Books

Textbooks:

1. R. C. Gonzalez and R. E. Woods, “Digital Image Processing”, Pearson Education.

Reference Books:

1. B. Chanda and D. Dutta Mazumdar, “Digital Image Processing and Analysis”, PHI.
2. Fundamentals of Digital Image Processing, A. K. Jain, Pearson India Education.

Let's Begin



Original Image



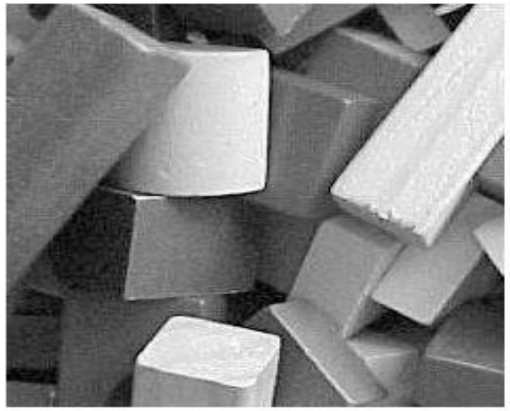
After Sharpening



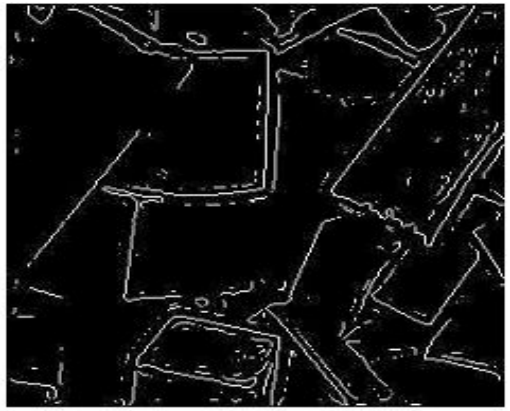
Original Image



After Removing Noise



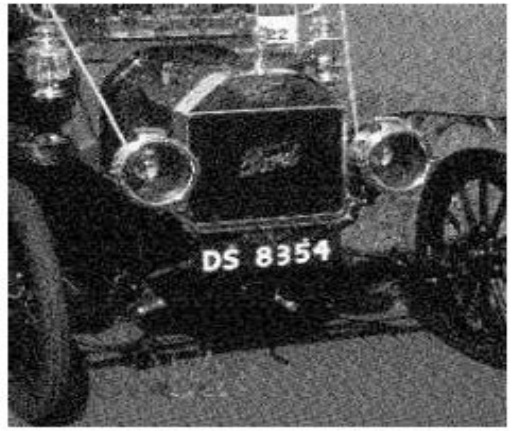
Original Image



Edge Image



Original Image



After removing blurness

What This Lab Covers

01 **Image Fundamentals**
Pixels, f(x,y), M×N matrices, storage & formats

02 **Read • Display • Save**
imread / imshow / imwrite

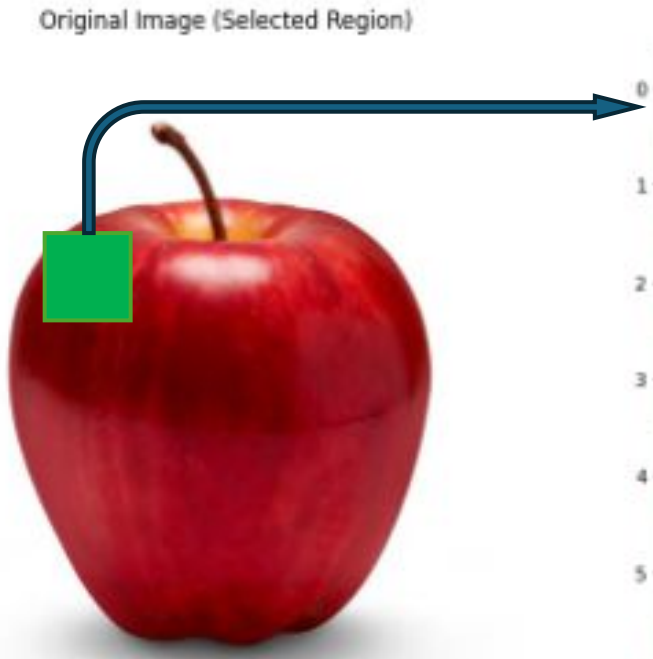
03 **Resize & Crop**
Scaling dimensions and array slicing

04 **Grayscale & ROI**
Color conversion and bounding boxes

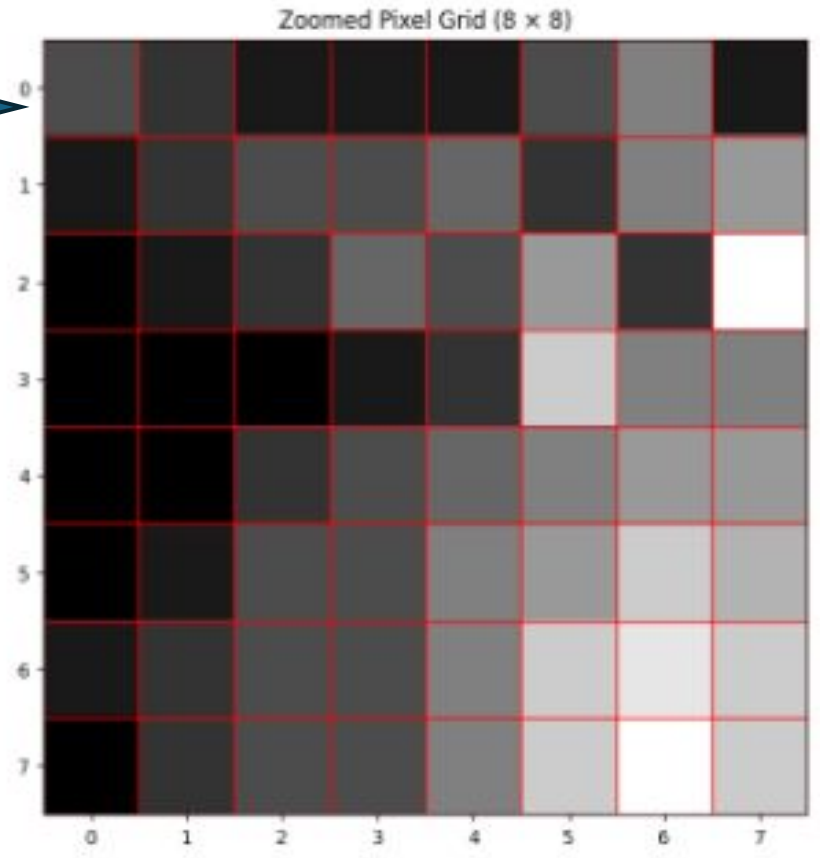
05 **Rotation & Scaling**
Affine transforms and interpolation

06 **RGB Channels**
Splitting and merging colour planes

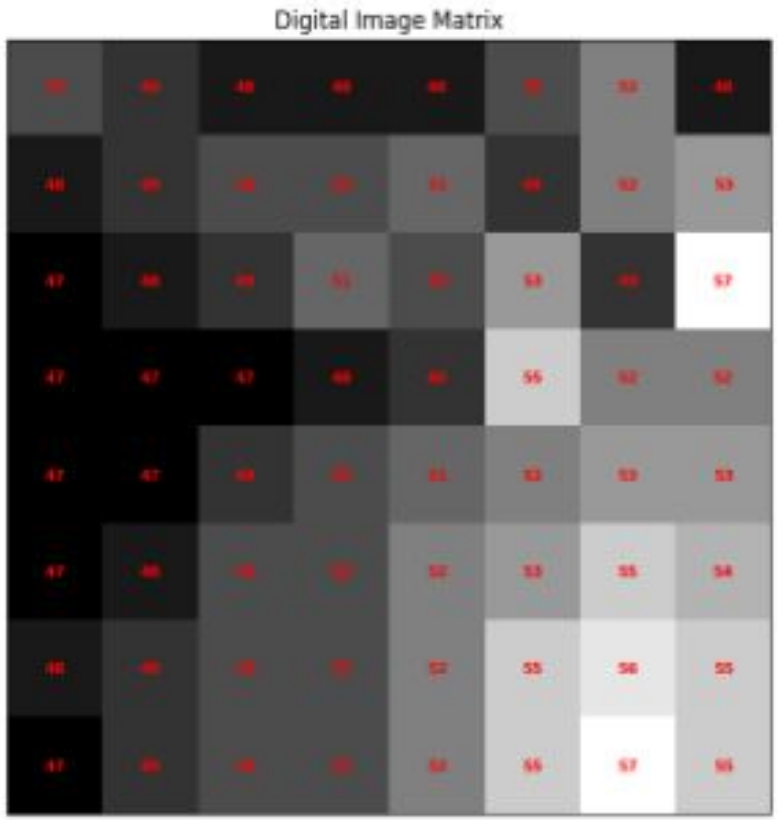
Introduction to a Digital Image



(a) Original Image: Selected Region of Interest (ROI)



(b) Zoomed Pixel Grid: Magnified Pixel Representation



(c) Digital Image Matrix: Pixel Intensity Matrix

What Is a Digital Image?

Definition

A digital image is a 2-D function f(x, y) where x, y are spatial coordinates (pixel location) and f is the intensity (brightness) at that location.

Discrete

Both coordinates and intensities are finite → the image becomes a digital M × N matrix.

Pixel

Each element f(i, j) is one pixel; an M × N image has M·N pixels.

f(x, y)

intensity at spatial location (x, y)

Storage

Storage = M × N × k bits (k = bits per pixel)

Gray image

1 intensity matrix

Colour image

3 matrices
R • G • B

Common Image Format

Raster formats (pixel-based)

JPEG `.jpg`

Lossy compression, widely used

PNG `.png`

Lossless, supports transparency

BMP `.bmp`

Uncompressed bitmap

TIFF `.tif`

High quality, layers & metadata

GIF `.gif`

256 colours, supports animation

Tools & Libraries: Python vs MATLAB

Python (open-source ecosystem)

OpenCV (cv2) — Fast processing & computer vision — NumPy BGR arrays

PIL / Pillow — Simple loading & manipulation

Matplotlib — Visualisation & plotting (RGB)

scikit-image — Research-grade processing (float arrays)

imageio — Flexible multi-format I/O

MATLAB (Image Processing Toolbox)

imread / imshow — Load & display — RGB uint8 by default

imwrite — Save; format inferred from extension

imresize / imrotate — Resize & rotate with interpolation

rgb2gray — Colour → grayscale conversion

insertShape / rectangle — Draw ROI boxes & annotations

Installing the Tools & Libraries

Python — pip

1. Install Python 3.x from python.org.
2. Install each library with pip.
3. Confirm the install by importing the package.

Python

```
pip install opencv-python numpy
pip install matplotlib pillow
pip install scikit-image imageio
# verify the installation
python -c "import cv2;
print(cv2.__version__)"
```

MATLAB — MathWorks installer

1. Install MATLAB via the MathWorks installer (a valid licence is required).
2. Add the Image Processing Toolbox: Home tab → Add-Ons → Get Add-Ons.
3. Check that the toolbox is available before use.

MATLAB

```
% list all installed toolboxes
ver
% is the Image Processing Toolbox
present?
license('test','image_toolbox')
ver('images')
```

OpenCV lives in the cv2 pip package; MATLAB's imread / imshow / imresize / rgb2gray require the Image Processing Toolbox.

Reading an Image

Python: **using OpenCV**

cv2.imread( ) is used to load an image from a specific file path in the programming environment. It returns a NumPy array.

Syntax: **cv2.imread(filename, flags)**

Parameters:

filename: Path to the image file

flags (optional): How the image is read

MATLAB

```
img = imread('retina.png');   % RGB, uint8
```

Common flags Options:

Flag	Value	Description
cv2.IMREAD_COLOR	1	Loads a **color image** (default)
cv2.IMREAD_GRAYSCALE	0	Loads image in **grayscale**
cv2.IMREAD_UNCHANGED	-1	Loads image as such including alpha channel

Display an Image

Python: using OpenCV

cv2.imshow() is used to display an image to the console of the computer.

Syntax: ***cv2.imshow(window_name, image_file)***

Parameters:

- ***window_name***: Title of the display window
- ***image_file***: NumPy array (typically from cv2.imread())

Important Points:

- The window automatically fits to the image size.
- You can create as many windows as you wish, but with different window names.

Required Functions:

- cv2.waitKey(0) – Keeps window open until a key is pressedcv2.
- destroyAllWindows() – Closes all OpenCV windows

MATLAB

```
imshow(img);
title('Retina');
% figure window
auto-fits the
image size
```

Display an Image using Matplotlib

plt.imshow() is used to display an image to the console of the computer.

Syntax:

import matplotlib.pylot as plt

plt.imshow(X, cmap=None, interpolation=None)

Parameters:

- ***X***: Image data (2D for grayscale or 3D for RGB)
  – Usually a NumPy array (e.g., loaded with OpenCV, PIL, or image processing libs)
- ***cmap***: Colormap (for grayscale or heatmaps), e.g., 'gray', 'viridis', etc.
- ***Interpolation***: Controls how pixels are interpolated: 'nearest', 'bilinear', etc.

*matplotlib.pyplot() takes input image argument in RGB format whereas cv2.imshow() takes input in BGR format.

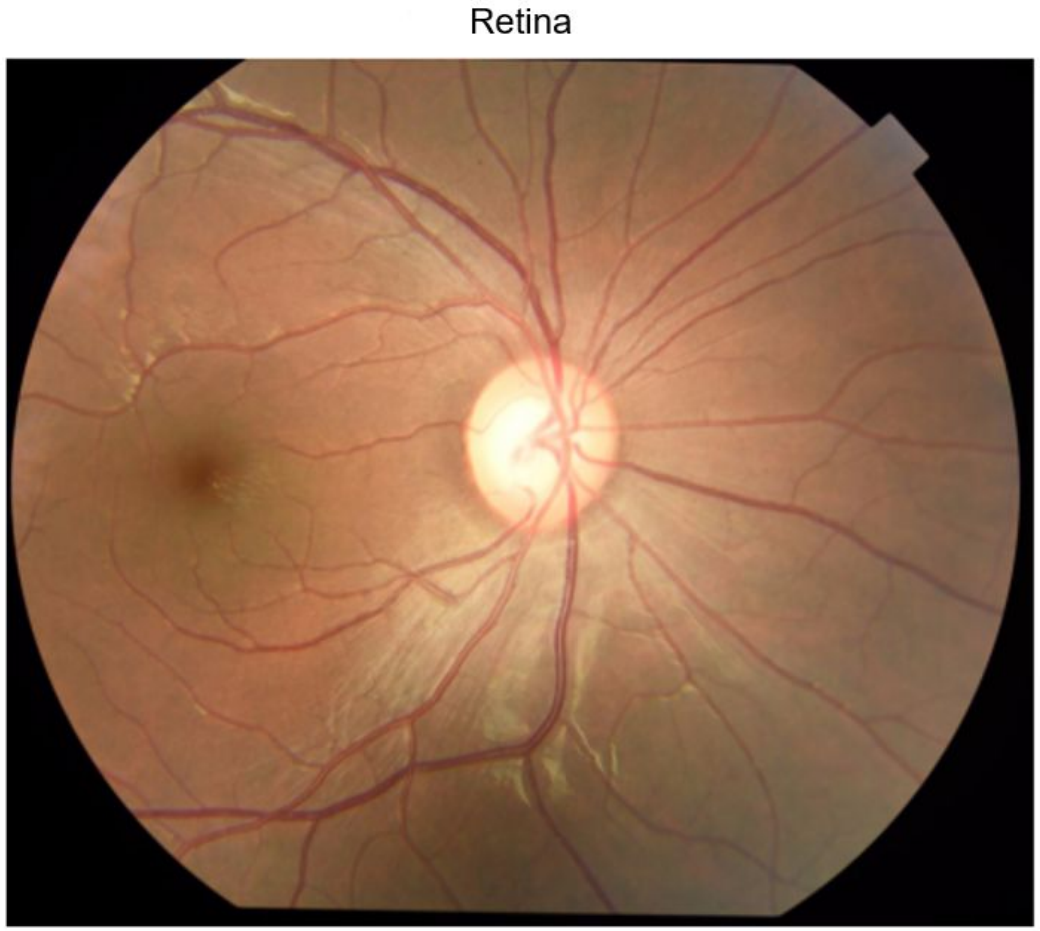
Example

```
import cv2
img = cv2.imread('retina.png', cv2.IMREAD_COLOR)
if img is not None:
    cv2.imshow('Retina', img)
    cv2.waitKey(0)
    cv2.destroyAllWindows()
else:
    print("Failed to load image.")
```

*** Important Points about imread() function:**

- Returns a **NumPy array** of shape:
- `(H, W, 3)` for color image (in **BGR** order)
- `(H, W)` for grayscale image
- OpenCV reads images as **8-bit unsigned integers (`uint8`)** by default.
- So each pixel value ranges from **0 to 255**, where:
- `0` = black
- `255` = white (for grayscale), or full intensity in RGB/BGR
- Returns `None` if image is not found or path is invalid

Output



- cv2.waitKey(0)
  - Keeps OpenCV image window open so you can view the image
  - Allows user interaction before the window closes

- cv2.destroyAllWindows()
  - Cleans up and frees system resources
  - Ensures no unwanted windows remain open

Writing / Saving an Image

Python

cv2.imwrite() is used to saves an image (numpy array) to a file on the computer.

Syntax: ***cv2.imwrite(filename, image_data)***

Parameters:

- ***filename***: Path and name of the output file (e.g., "output.jpg", "result.png")
- ***image_data:*** The image data to save (e.g., numpy array)

Return Value: Returns True if the image is saved successfully, False otherwise

```
import cv2
img = cv2.imread("input.jpg")
cv2.imwrite("output.png", img)
```

MATLAB

```
imwrite(img,'output.png');

% format inferred from the
file extension
```

Example: Save the image “input.jpg” with the new name “output.jpg” to CWD

Resizing an Image

Python

cv2.resize() function is used to change the dimensions of an image (scale up or down)

Syntax: ***cv2.resize(source, dsize, dest, fx, fy, interpolation)***

Parameters:

- ***source*** **:** Input Image array (Single-channel, 8-bit or floating-point)
- ***dsize*** **:** Size of the output array
- ***dest*** **:** Output array (Similar to the dimensions and type of Input image array) [optional]
- ***fx*** **:** Scale factor along the horizontal axis [optional]
- ***fy*** **:** Scale factor along the vertical axis [optional]
- ***interpolation*** **:** One of the interpolation methods [optional], default is Linear interpolation

MATLAB

```
resized = imresize(img,
[200, 200]);
% [rows cols] order in
MATLAB
imshow(resized);
```

Resizing an Image

Sample Code

```
img = cv2.imread('input.jpg')

resized = cv2.resize(img, (200,
200))

cv2.imshow('Resized Image',
resized)
```

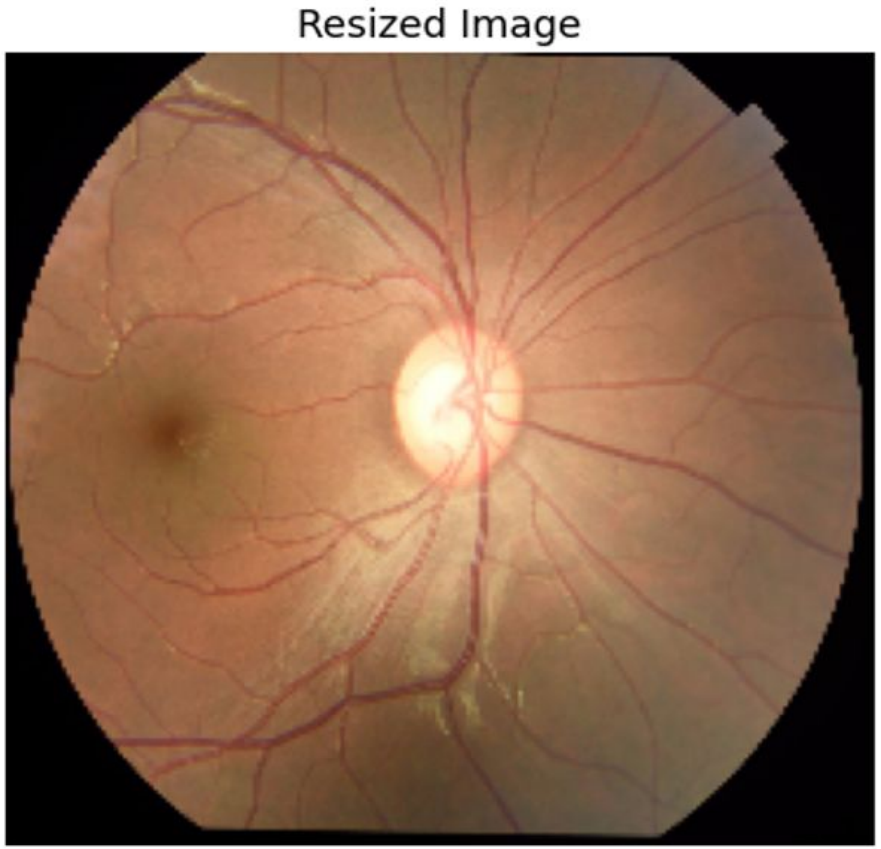


Image Cropping

Python

- Image cropping is done using slicing.
  Syntax: ***array[start_index:stop_index:step]***

- For 2D image cropping slicing can be done by:
  array[row_start:row_stop:row_step, col_start:col_stop:col_step]

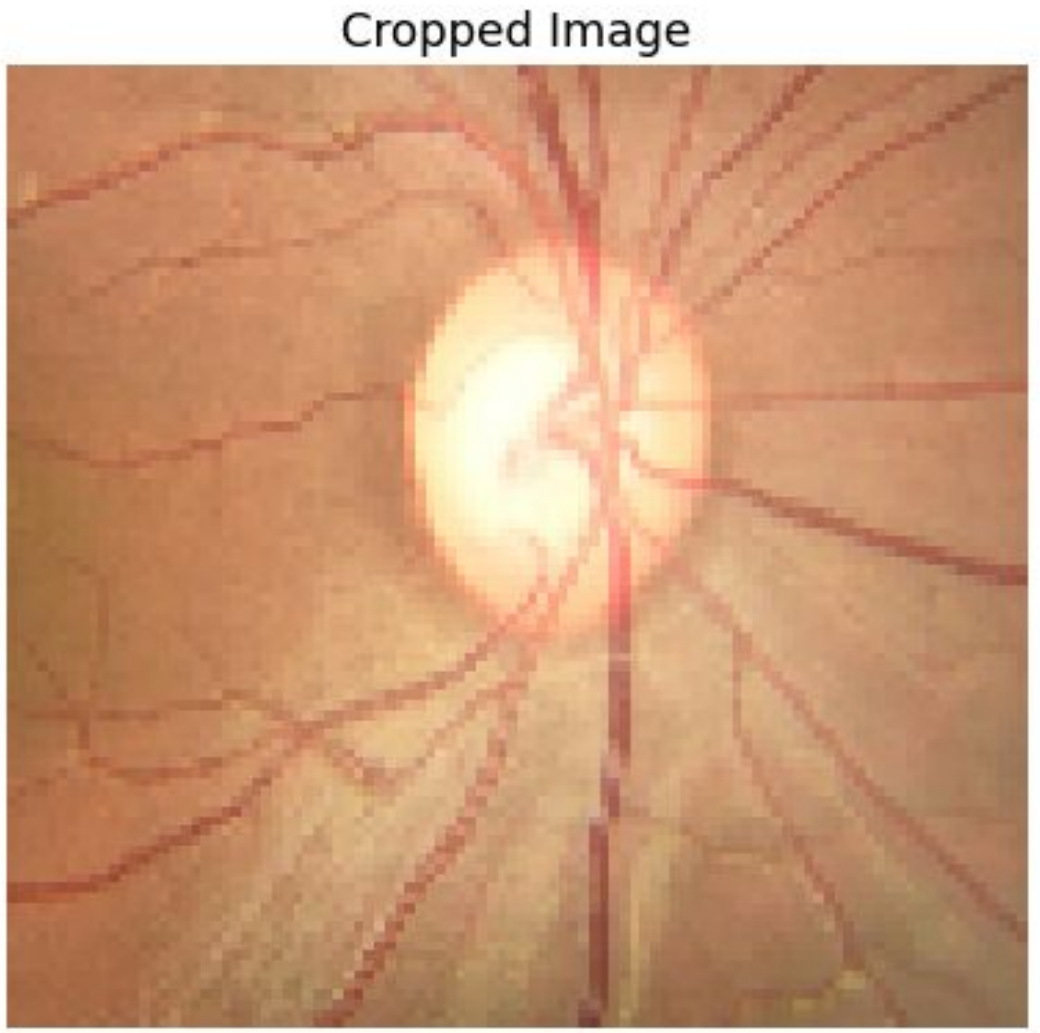
MATLAB

```
[h, w, ~] = size(img); % Get image height and width; ignore the number of color channels
crop = img(1:floor(h/2), 1:w, :); % top half
imshow(crop);
% MATLAB indices start at 1, not 0
```

Image Cropping

Sample code to crop 50×50 image from top right corner:

```
img = cv2.imread('retina.png',
cv2.IMREAD_COLOR)
img_cropped =
img[50:150,50:150]
cv2.imshow('Cropped Image',
img_cropped)
cv2.waitKey(0)
cv2.destroyAllWindows()
```



Convert an Image to Grayscale

Python

OpenCV function:

cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

- The ***cv2.cvtColor*** function converts a 3-channel BGR image to a single-channel grayscale image
- Uses formula: Y = 0.299*R + 0.587*G + 0.114*B

- Displays using matplotlib:
  - Use cmap='gray' to show intensity.
  - Removes color interpretation in display.

MATLAB

```
gray = rgb2gray(img);
imshow(gray);
% use luma weighting
```

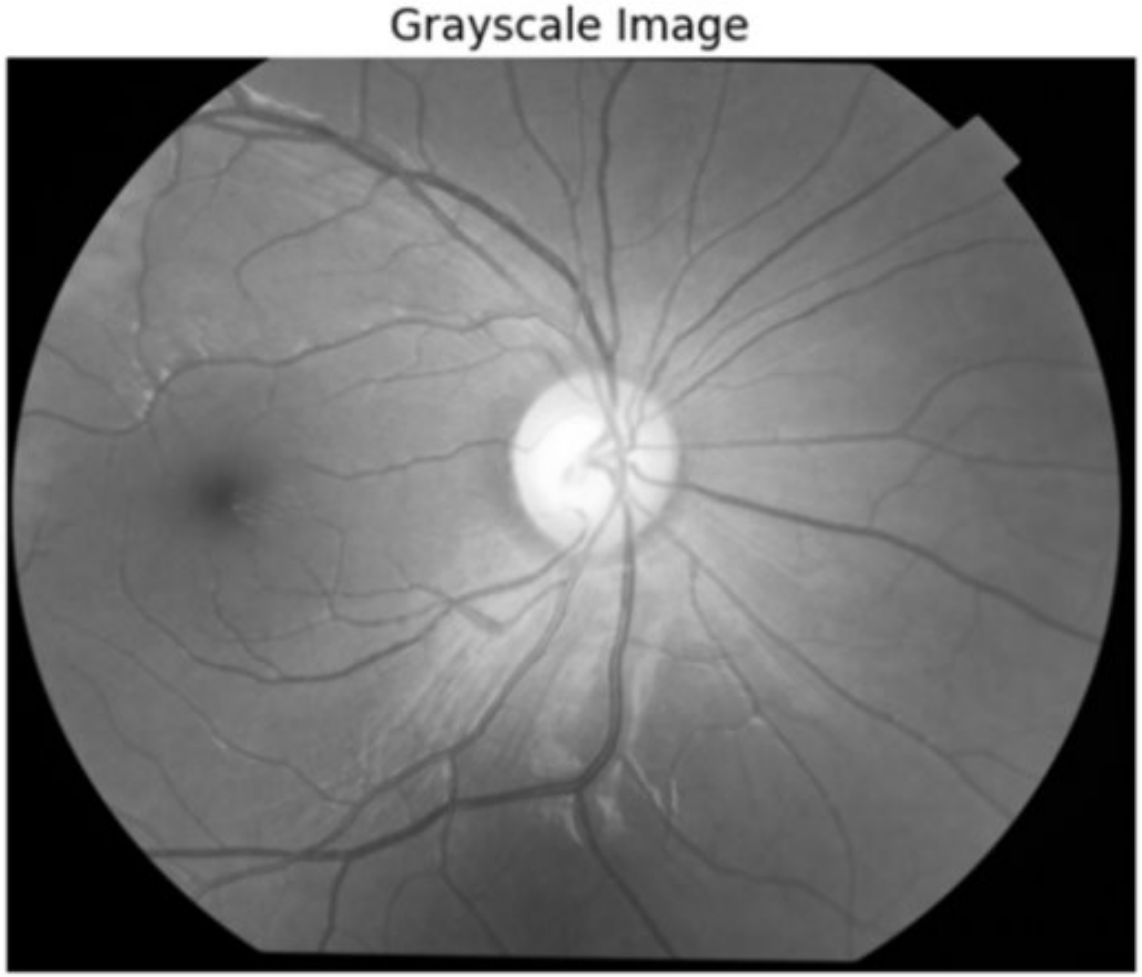
Convert an Image to Grayscale

Sample Code:

```
gray = cv2.cvtColor(img,
cv2.COLOR_BGR2GRAY)

plt.imshow(gray, cmap='gray’)

plt.title('Grayscale Image')
plt.axis('off')
plt.show()
```



Plot a Bounding Box

Python

Use **cv2.rectangle()** to draw rectangles on images.

Syntax:

cv2.rectangle(image, start_point, end_point, color, thickness)

In this example:

start_point = (50, 50)
end_point = (150, 150)
color = (0, 255, 0) → Green in BGR format
thickness = 2 → Border width in pixels

MATLAB

```
img = insertShape(img,'Rectangle',
[x1 y1 w h], 'Color','green',
'LineWidth',6);
imshow(img);
% x1 = 150; X-coordinate of the
top-left corner
% y1 = 100; Y-coordinate of the
top-left corner
% w  = 300; Rectangle width
% h  = 200; Rectangle height
```

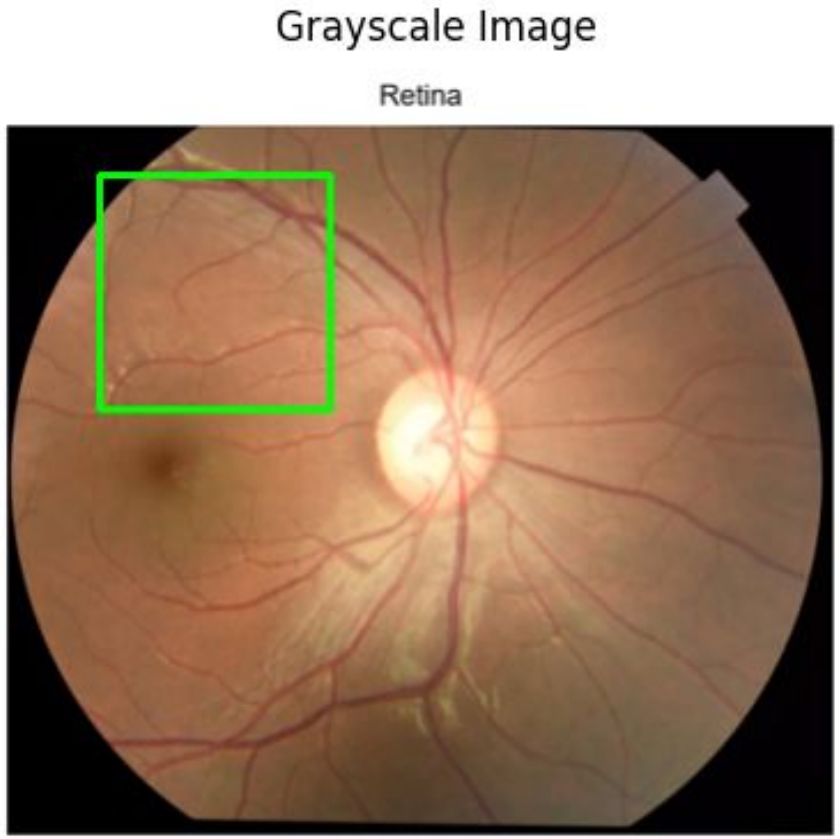
Plot a Bounding Box

Sample Code

```
cv2.rectangle(img_copy, (50, 50),
(150, 150), (0, 255, 0), 2)

plt.imshow(cv2.cvtColor(img_copy,
cv2.COLOR_BGR2RGB))

plt.title('Grayscale Image')
plt.axis('off')
plt.show()
```



Rotation of an Image

Python

Use **cv2.getRotationMatrix2D()** to compute the affine transformation matrix for rotation

Syntax:

cv2.getRotationMatrix2D(center, angle, scale)

In this example:

center = (w // 2, h // 2) → Image center

angle = 45 degrees

scale = 1.0 (no zoom)

Then apply the rotation using:

rotated = cv2.warpAffine(img, M, (w, h))

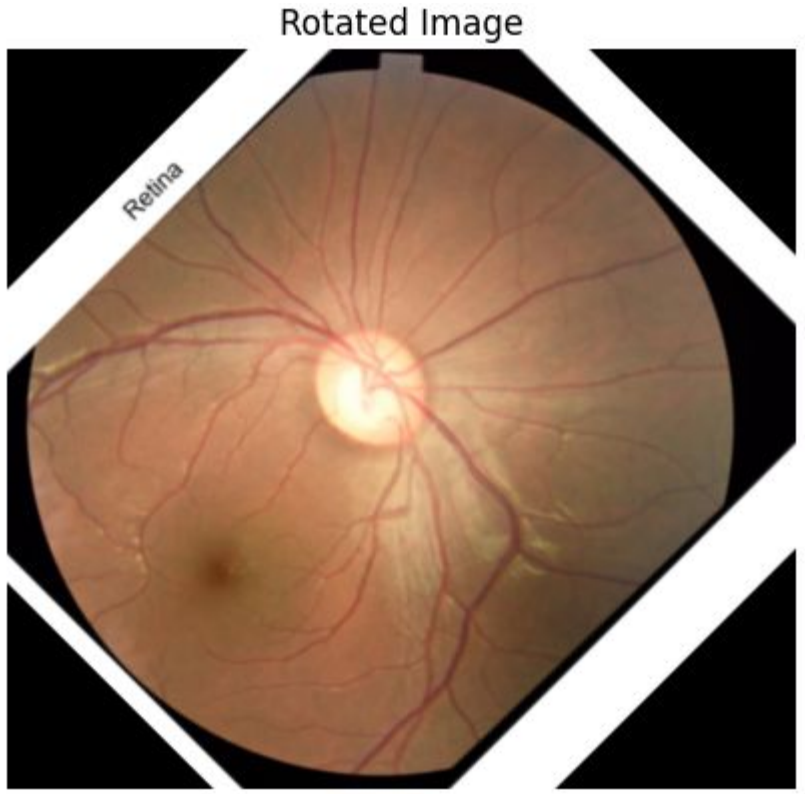
MATLAB

```
rot = imrotate(img, 45); % Rotate image 45° counterclockwise (CCW)
imshow(rot); % Display rotated image
```

Rotation of an Image

Sample Code:

```
(h, w) = img.shape[:2]
center = (w // 2, h // 2)
M = cv2.getRotationMatrix2D(center, 45, 1.0)
rotated = cv2.warpAffine(img, M, (w, h))
plt.imshow(cv2.cvtColor(rotated,
cv2.COLOR_BGR2RGB))
plt.title('Rotated Image')
plt.axis('off')
plt.show()
```



Scaling of an Image

Python

Use **cv2.resize()** to scale images up or down

Syntax:

scaled = cv2.resize(img_gray, None, fx=1.5, fy=1.5, interpolation=cv2.INTER_LINEAR)

Parameters:

fx = 1.5: Scale width by 1.5
fy = 1.5: Scale height by 1.5
interpolation = cv2.INTER_LINEAR: Smooth interpolation for enlarging

MATLAB

```
scaled = imresize(gray, 1.5, 'bilinear');
imshow(scaled);
% scale factor enlarges by 1.5x
```

Scaling of an Image

Sample Code

```
scaled = cv2.resize(img_gray, None, fx=1.5, fy=1.5, interpolation=cv2.INTER_LINEAR)

plt.imshow(scaled, cmap='gray')
plt.title('Scaled Image')
plt.axis('off')
plt.show()
```





Splitting the RGB Channels

Python

- Use ***cv2.split(img)*** to separate a color image into its individual B, G, R components.
- OpenCV loads images in **BGR** format (Blue-Green-Red)

After splitting:

R: Red channel → plt.imshow(R, cmap='gray')
G: Green channel → plt.imshow(G, cmap='gray')
B: Blue channel → plt.imshow(B, cmap='gray')

- Each channel is displayed as a grayscale image, showing intensity in that color.
- plt.subplot() is used to display all three channels side-by-side.
- plt.tight_layout() adjusts spacing for better visual alignment.

MATLAB

```
R = img(:,:,1);
G = img(:,:,2);
B = img(:,:,3);
imshow(R);    % red
plane
```

Splitting the RGB Channels

Sample Code:

```
import cv2
import matplotlib.pyplot as plt
# load original image
img = cv2.imread('Retina.jpg')  # Loads in BGR format

# Split into B, G, R channels
B, G, R = cv2.split(img)

plt.imshow(R, cmap='gray')
plt.title('Red Channel')
plt.axis('off')
plt.show()
```



Merging the RGB channels

Python

- After splitting channels using cv2.split(), you can reconstruct the image using cv2.merge().

Syntax:

merged = cv2.merge([B, G, R])

- Channels must be merged in **BGR order** to match OpenCV's default format.

- This restores the original color image from separate channel components.

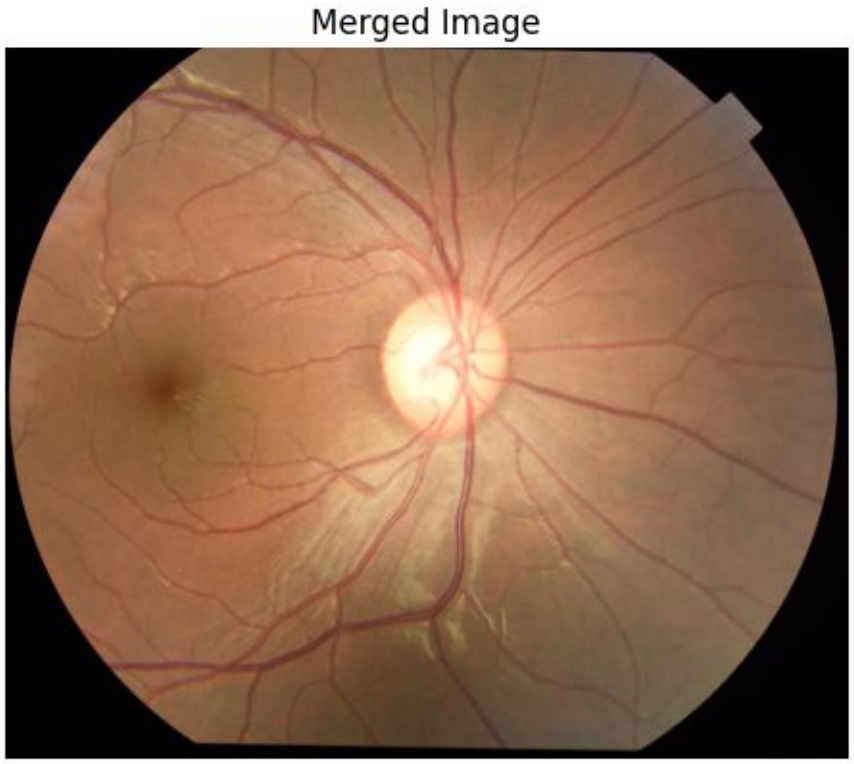
MATLAB

```
merged = cat(3, R, G, B);
imshow(merged);
% cat(3,...) stacks planes
along dim 3
```

Merging the RGB channels

Sample Code

```
merged = cv2.merge([B, G, R])  # Reconstruct in
BGR order
# Convert to RGB for display
plt.imshow(cv2.cvtColor(merged,
cv2.COLOR_BGR2RGB))
plt.title('Merged Image')
plt.axis('off')
plt.show()
```



Python ↔ MATLAB Quick Reference

Operation	Python (OpenCV)	MATLAB
Read image	`cv2.imread(path)`	`imread(path)`
Display	`cv2.imshow / plt.imshow`	`imshow(img)`
Save	`cv2.imwrite(path, img)`	`imwrite(img, path)`
Resize	`cv2.resize(img,(w,h))`	`imresize(img,[r c])`
Crop	`img[r0:r1, c0:c1]`	`img(r0:r1, c0:c1, :)`
Grayscale	`cv2.cvtColor(...BGR2GRAY)`	`rgb2gray(img)`
Rectangle	`cv2.rectangle(...)`	`insertShape(...,'Rectangle')`
Rotate	`warpAffine + getRotationMatrix2D`	`imrotate(img, ang)`
Split channels	`cv2.split(img)`	`img(:,:,k)`
Merge channels	`cv2.merge([B,G,R])`	`cat(3, R,G,B)`

Assignments

1. Write a Python program to read the "**retina.png**" image and resize it to 200 × 200 pixels then perform the following tasks:

   a) Draw a blue rectangular region of interest(ROI) between the points (80,80), (80,160), (160,80), and (160, 160) with a thickness of 5 pixels. Then display the ROI marked image using matplotlib.

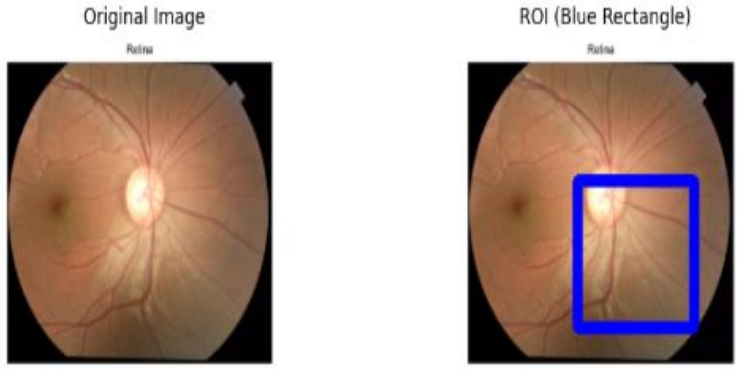
   b) Vertically crop the image into two equal halves, then save both images.

   c) Horizontally crop the images into four equal parts, then save all the images.

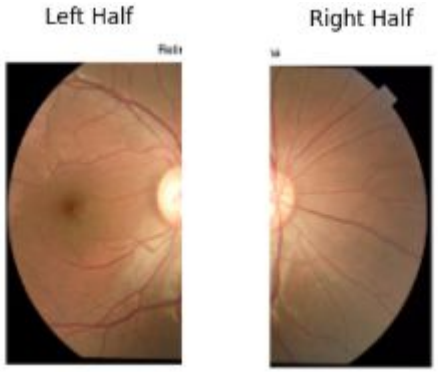
   d) Convert the image into a gray scale image, then draw a white straight line from the point (80,80) to the point (160,80), then display the image.

Assignments Sample Output

1(a): Output



1(b): Output



1(c) : Output



1(d): Output



Thank You

Questions & Discussion?

Shiv Sagar
Teaching Assistant
Department of Computer Science & Engineering
IIT (ISM) Dhanbad